Programming with Python (2501IT42)

Department of Freshman Engineering

Detailed 10-Mark Answers – UNIT 3, UNIT 4, UNIT 5

UNIT – 3 : FUNCTIONS, MODULES AND PACKAGES

1(a) Define Function. Explain Function Creation and Function Calling in Python with an Example Program.

Definition of Function

A function is a reusable block of code that performs a specific task. Functions help in dividing a large program into smaller manageable parts. They improve readability, reduce code duplication, and simplify debugging.

Functions are executed only when they are called.

Advantages of Functions

1. Code reusability
2. Reduces repetition of code
3. Improves readability
4. Simplifies debugging and testing
5. Makes program modular
6. Easier maintenance of programs

Function Creation in Python

Functions are created using the `def` keyword.

Syntax

```
def function_name(parameters):
    statements
    return value
```

Components of a Function

1. `def` → Keyword used to define a function
2. `function_name` → Name of the function
3. `parameters` → Inputs passed to the function
4. `:` → Indicates start of function body
5. `return` → Sends result back to caller

Function Calling

Calling a function means executing the function whenever required.

Syntax

```
function_name(arguments)
```

Arguments are values passed to the function during calling.

Example Program

```
# Function definition

def add(a, b):
    result = a + b
    return result

# Function call
x = 10
y = 20
sum = add(x, y)

print("Sum =", sum)
```

Explanation of Program

1. `def add(a,b)` creates a function named `add`
2. `a` and `b` are parameters
3. `result = a+b` adds two numbers
4. `return result` returns value to caller
5. `add(x,y)` calls the function
6. Returned value is stored in `sum`
7. Output is displayed using `print()`

Output

```
Sum = 30
```

Conclusion

Functions are important in Python programming because they provide modularity, reusability, and better organization of code.

==========

1(b) Explain Local and Global Variables with Suitable Examples

Local Variable

A variable declared inside a function is called a local variable.

Characteristics

1. Created inside the function
2. Accessible only within the function
3. Destroyed after function execution
4. Cannot be accessed outside the function

Example of Local Variable

```
def display():
    x = 100
```

```
    print("Inside function:", x)


display()
```

Output

```
Inside function: 100
```

In this example, `x` is a local variable.

Global Variable

A variable declared outside all functions is called a global variable.

Characteristics

1. Declared outside functions
2. Accessible throughout the program
3. Can be used inside and outside functions
4. Exists until program termination

Example of Global Variable

```
x = 50


def show():
    print("Inside function:", x)


show()
print("Outside function:", x)
```

Output

```
Inside function: 50
Outside function: 50
```

Using global Keyword

If we want to modify a global variable inside a function, we use the `global` keyword.

Example

```
x = 10


def change():
    global x
    x = x + 5
    print("Inside function:", x)


change()
print("Outside function:", x)
```

Output

```
Inside function: 15
Outside function: 15
```

Difference Between Local and Global Variables

Local Variable	Global Variable
Declared inside function	Declared outside function
Accessible only inside function	Accessible throughout program
Temporary existence	Exists until program ends
Cannot be accessed globally	Can be accessed anywhere

Conclusion

Local variables provide data security within functions, while global variables help in sharing data across functions.

==========

2(a) Explain Different Function Arguments with Suitable Examples

Introduction

Arguments are values passed to functions during function calling.

Python supports different types of arguments.

Types of Function Arguments

1. Positional Arguments
2. Keyword Arguments
3. Default Arguments
4. Variable Length Arguments

---

1. Positional Arguments

Arguments are assigned according to their position.

Example

```
def student(name, age):
    print("Name:", name)
    print("Age:", age)


student("Rahul", 20)
```

Output

```
Name: Rahul
Age: 20
```

---

2. Keyword Arguments

Arguments are passed using parameter names.

Example

```
def employee(id, name):
    print("ID:", id)
    print("Name:", name)


employee(name="Kiran", id=101)
```

Output

```
ID: 101
Name: Kiran
```

---

3. Default Arguments

A default value is assigned to parameters.

Example

```
def greet(name="Student"):
    print("Hello", name)


greet()
greet("Arun")
```

Output

```
Hello Student
Hello Arun
```

---

4. Variable Length Arguments

Used when number of arguments is unknown.

`*args` is used.

Example

```
def total(*numbers):
    sum = 0
    for i in numbers:
        sum = sum + i
    print("Sum =", sum)


total(10, 20)
total(1, 2, 3, 4)
```

Output

```
Sum = 30
Sum = 10
```

Conclusion

Different types of arguments provide flexibility in passing values to functions.

==========

2(b) Write a Python Function that Takes Two Lists and Returns True if They Have at Least One Common Member

Program

```
def common_data(list1, list2):
    for i in list1:
        if i in list2:
            return True
    return False


list1 = [1, 2, 3, 4]
list2 = [5, 6, 3, 8]

print(common_data(list1, list2))
```

Explanation

1. Function `common_data()` accepts two lists
2. Loop checks each element of first list
3. `if i in list2` checks membership
4. If common element exists, returns `True`
5. Otherwise returns `False`

Output

```
True
```

Conclusion

The program efficiently checks whether two lists contain common elements.

==========

3(a) Explain Anonymous Functions with Examples

Definition

Anonymous functions are functions without names.

In Python, anonymous functions are created using the `lambda` keyword.

They are also called lambda functions.

Syntax

```
lambda arguments : expression
```

Characteristics of Lambda Functions

1. No function name

2. Small and simple functions
3. Contains single expression
4. Returns value automatically
5. Useful for temporary operations

Example 1 – Addition

```
sum = lambda a, b: a + b

print(sum(10, 20))
```

Output

```
30
```

Example 2 – Square of a Number

```
square = lambda x: x * x

print(square(5))
```

Output

```
25
```

Example 3 – Using Lambda with map()

```
numbers = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, numbers))

print(result)
```

Output

```
[2, 4, 6, 8]
```

Advantages

1. Reduces code size
2. Easy for small operations
3. Improves readability in short functions
4. Useful with functions like map(), filter(), reduce()

Conclusion

Lambda functions provide a concise way to create small anonymous functions.

==========

3(b) Explain Following Keywords with Suitable Examples

i) import ii) from iii) as

1. import Keyword

`import` is used to include an entire module in a program.

Syntax

```
import module_name
```

Example

```
import math

print(math.sqrt(25))
```

Output

```
5.0
```

---

2. from Keyword

`from` is used to import specific functions from a module.

Syntax

```
from module_name import function_name
```

Example

```
from math import sqrt

print(sqrt(36))
```

Output

```
6.0
```

---

3. as Keyword

`as` is used to create an alias name.

Syntax

```
import module_name as alias_name
```

Example

```
import math as m

print(m.factorial(5))
```

Output

```
120
```

Conclusion

These keywords help in importing and using Python modules efficiently.

==========

4(a) List and Explain Standard Modules in Python

Definition

Standard modules are pre-defined modules provided by Python.

These modules contain functions and classes for various operations.

Common Standard Modules

1. math Module

Provides mathematical functions.

Example

```
import math
print(math.sqrt(16))
```

Functions:

- sqrt()
- factorial()
- sin()
- cos()

---

2. random Module

Used for generating random numbers.

Example

```
import random
print(random.randint(1, 10))
```

Functions:

- randint()
- random()
- choice()

---

3. datetime Module

Used for date and time operations.

Example

```
import datetime
print(datetime.datetime.now())
```

---

4. os Module

Provides operating system related functions.

Example

```
import os
print(os.getcwd())
```

Functions:

- getcwd()
- mkdir()
- remove()

---

5. sys Module

Provides system-specific functions.

Example

```
import sys
print(sys.version)
```

Advantages of Standard Modules

1. Saves programming time
2. Reduces code complexity
3. Provides reusable functions
4. Improves efficiency

Conclusion

Python standard modules provide ready-made functionalities for different programming tasks.

==========

4(b) Differentiate Top-Down and Bottom-Up Approaches with Examples

Top-Down Approach

In top-down approach, the problem is divided into smaller subproblems.

Design starts from main module and proceeds downward.

Characteristics

1. Starts from main problem
2. Uses step-by-step refinement
3. Easier planning
4. Functional decomposition is used

Example

ATM System:

1. Main ATM System
2. Balance Check Module
3. Cash Withdrawal Module
4. Deposit Module

Bottom-Up Approach

In bottom-up approach, smaller modules are developed first and then combined.

Characteristics

1. Starts from small modules
2. Modules are integrated later
3. Reusability is high
4. Common in object-oriented programming

Example

Library System:

1. Book Module
2. Student Module
3. Issue Module
4. Combined into Library System

Difference Between Top-Down and Bottom-Up

Top-Down	Bottom-Up
Starts from main problem	Starts from smaller modules
Functional approach	Object-oriented approach
Refinement method	Integration method
Less reusable	Highly reusable

Conclusion

Both approaches are useful in software development depending on project requirements.

==========

5(a) Explain the Advantage of Packages. Explain the Procedure to Create a Package and Use the Same.

Definition of Package

A package is a collection of modules organized in directories.

Packages help in organizing Python programs systematically.

Advantages of Packages

1. Organizes large programs
2. Avoids naming conflicts
3. Improves modularity
4. Simplifies maintenance
5. Encourages code reuse
6. Provides hierarchical structure

Procedure to Create a Package

Step 1: Create Folder

Create a folder named `mypackage`

Step 2: Create init.py File

Inside folder create:

```
__init__.py
```

This file indicates the folder is a package.

Step 3: Create Module

Create file `calc.py`

```
def add(a, b):
    return a + b
```

Step 4: Use Package

```
from mypackage.calc import add

print(add(10, 20))
```

Output

```
30
```

Conclusion

Packages improve organization, modularity, and reusability in Python applications.

==========

5(b) Define Recursion. Explain Recursive Functions with Suitable Examples.

Definition

Recursion is a process in which a function calls itself repeatedly until a stopping condition is satisfied.

A function that calls itself is called a recursive function.

Components of Recursion

1. Base Condition
2. Recursive Call

Example – Factorial Program

```
def factorial(n):
    if n == 1:
        return 1
    else:
        return n * factorial(n - 1)


print(factorial(5))
```

Explanation

1. `factorial(5)` calls `factorial(4)`
2. `factorial(4)` calls `factorial(3)`
3. Process continues until `n == 1`
4. Then values return back

Calculation:

5 × 4 × 3 × 2 × 1 = 120

Output

```
120
```

Advantages

1. Simplifies complex problems
2. Reduces code size
3. Useful for tree and graph problems

Disadvantages

1. More memory usage
2. Slower execution
3. May cause stack overflow

Conclusion

Recursive functions provide elegant solutions for repetitive problems.

==========

6(a) Explain Different Function Categories with Examples

Function Categories in Python

1. Built-in Functions
2. User-defined Functions
3. Anonymous Functions
4. Recursive Functions

---

1. Built-in Functions

Functions already provided by Python.

Example

```
print(len("Python"))
```

---

2. User-defined Functions

Functions created by programmer.

Example

```
def add(a, b):
    return a + b

print(add(5, 10))
```

---

3. Anonymous Functions

Functions without names using lambda.

Example

```
square = lambda x: x * x
print(square(4))
```

---

4. Recursive Functions

Functions calling themselves.

Example

```
def fact(n):
    if n == 1:
        return 1
    return n * fact(n - 1)

print(fact(4))
```

Conclusion

Different categories of functions provide flexibility and efficiency in programming.

==========

6(b) Explain Top-Down and Bottom-Up Approaches

Top-Down Approach

The entire problem is divided into smaller modules.

Features

1. Starts from main module
2. Uses decomposition
3. Easier design

4. Better planning

Example

Online Shopping System:

- Login Module
- Payment Module
- Order Module

Bottom-Up Approach

Small modules are developed first and combined.

Features

1. Starts from low-level modules
2. Integration based
3. Reusability is high
4. Used in OOP

Example

Banking System:

- Customer Module
- Account Module
- Transaction Module

Comparison

Top-Down	Bottom-Up
Main problem first	Small modules first
Functional approach	Object-oriented approach
Stepwise refinement	Module integration

Conclusion

Both approaches help in systematic software development.

==========

UNIT – 4 : OBJECT ORIENTED PROGRAMMING AND FILE HANDLING

1(a) Define Inheritance and Explain Types of Inheritance with Examples

Definition of Inheritance

Inheritance is an object-oriented programming feature in which one class acquires properties and methods of another class.

The existing class is called base class or parent class. The new class is called derived class or child class.

Advantages of Inheritance

1. Code reusability
2. Reduces redundancy
3. Improves maintainability
4. Supports hierarchical classification

Types of Inheritance

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

---

1. Single Inheritance

One child class inherits one parent class.

Example

```
class Parent:
    def show(self):
        print("Parent class")

class Child(Parent):
    pass

c = Child()
c.show()
```

---

2. Multiple Inheritance

One child class inherits multiple parent classes.

Example

```
class A:
    def display1(self):
        print("Class A")

class B:
    def display2(self):
        print("Class B")

class C(A, B):
    pass

obj = C()
obj.display1()
obj.display2()
```

---

3. Multilevel Inheritance

A class inherits another derived class.

Example

```
class A:
    def show(self):
        print("Class A")
```

```
class B(A):
    pass

class C(B):
    pass

obj = C()
obj.show()
```

---

4. Hierarchical Inheritance

Multiple child classes inherit one parent class.

Example

```
class Parent:
    def show(self):
        print("Parent")

class Child1(Parent):
    pass

class Child2(Parent):
    pass
```

---

5. Hybrid Inheritance

Combination of multiple inheritance types.

Conclusion

Inheritance improves code reusability and supports hierarchical relationships.

==========

1(b) Demonstrate Polymorphism Using an Example Python Program

Definition of Polymorphism

Polymorphism means “many forms”.

Same method behaves differently for different objects.

Types of Polymorphism

1. Method Overloading
2. Method Overriding

Example Program

```
class Bird:
    def sound(self):
        print("Bird makes sound")

class Dog(Bird):
    def sound(self):
        print("Dog barks")

class Cat(Bird):
    def sound(self):
        print("Cat meows")


obj1 = Dog()
obj2 = Cat()

obj1.sound()
obj2.sound()
```

Output

```
Dog barks
Cat meows
```

Explanation

Different classes use same method name `sound()` with different behaviors.

Advantages

1. Flexibility
2. Code extensibility
3. Improves readability

Conclusion

Polymorphism enables one interface to represent different behaviors.

==========

2(a) Demonstrate Operator Overloading with an Example Program

Definition

Operator overloading allows operators to perform different operations depending on operands.

Python uses special methods for operator overloading.

Example: `__add__()` for `+`

Example Program

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value


n1 = Number(10)
n2 = Number(20)
```

```
print(n1 + n2)
```

Output

```
30
```

Explanation

1. `__add__()` overloads `+` operator
2. `n1 + n2` internally calls `__add__()`
3. Adds values of objects

Advantages

1. Improves readability
2. Makes objects behave like built-in types
3. Simplifies coding

Conclusion

Operator overloading enhances flexibility in object-oriented programming.

==========

2(b) Demonstrate File Operations with a Suitable Python Program

File Operations

Python supports various file operations such as:

1. Creating files
2. Reading files
3. Writing files
4. Appending files
5. Closing files

Example Program

```
# Writing into file
file = open("sample.txt", "w")
file.write("Hello Python")
file.close()

# Reading file
file = open("sample.txt", "r")
content = file.read()
print(content)
file.close()
```

Output

```
Hello Python
```

Explanation

1. `open()` opens file
2. `w` mode writes data
3. `write()` stores text
4. `r` mode reads data
5. `read()` retrieves contents
6. `close()` closes file

Conclusion

File handling enables permanent storage and retrieval of data.

==========

3(a) Demonstrate the Design with Classes Using a Case Study

Case Study – Bank Account System

Program

```
class Bank:
    def __init__(self, name, balance):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount

    def display(self):
        print("Name:", self.name)
        print("Balance:", self.balance)


obj = Bank("Rahul", 5000)
obj.deposit(2000)
obj.withdraw(1000)
obj.display()
```

Output

```
Name: Rahul
Balance: 6000
```

Explanation

1. `Bank` class stores account details
2. Constructor initializes values
3. Deposit method adds money
4. Withdraw method deducts money
5. Display method shows details

Conclusion

Classes help in designing real-world applications effectively.

==========

3(b) Explain the Functions write and writelines with Examples

write() Function

`write()` writes a string into a file.

Syntax

```
file.write(string)
```

Example

```
file = open("data.txt", "w")
file.write("Python Programming")
file.close()
```

---

writelines() Function

`writelines()` writes multiple strings into a file.

Syntax

```
file.writelines(list)
```

Example

```
file = open("data.txt", "w")
lines = ["Python\n", "Java\n", "C++\n"]
file.writelines(lines)
file.close()
```

Difference Between write() and writelines()

write()	writelines()
Writes single string	Writes multiple strings
Accepts string	Accepts list of strings

Conclusion

Both functions are useful for storing data into files.

==========

4(a) Demonstrate the Log File Writing in Python Program

Definition

Log files store program execution details, errors, and events.

Python uses `logging` module for logging.

Example Program

```
import logging

logging.basicConfig(filename='app.log', level=logging.INFO)

logging.info('Program started')
logging.warning('Warning message')
logging.error('Error occurred')

print("Log messages written")
```

Explanation

1. `basicConfig()` configures logging
2. `filename` specifies log file

3. `INFO`, `WARNING`, `ERROR` are log levels
4. Messages are stored in log file

Advantages

1. Helps debugging
2. Maintains execution records
3. Tracks errors

Conclusion

Logging is important for monitoring and debugging applications.

==========

4(b) Develop a Program to Read Config Files in Python

Example Program

```
import configparser

config = configparser.ConfigParser()
config.read('config.ini')

print(config['database']['host'])
print(config['database']['user'])
```

Sample config.ini File

```
[database]
host = localhost
user = admin
```

Explanation

1. `ConfigParser()` creates parser object

2. `read()` reads configuration file
3. Data accessed using section and key names

Advantages

1. Stores settings separately
2. Easy configuration management
3. Improves flexibility

Conclusion

Config files help manage application settings efficiently.

==========

5(a) Demonstrate read, readline, and readlines with Examples

1. read()

Reads entire file content.

Example

```
file = open("sample.txt", "r")
print(file.read())
file.close()
```

---

2. readline()

Reads one line at a time.

Example

```
file = open("sample.txt", "r")
print(file.readline())
file.close()
```

---

3. readlines()

Reads all lines and returns list.

Example

```
file = open("sample.txt", "r")
print(file.readlines())
file.close()
```

Difference

Function	Purpose
read()	Reads entire file
readline()	Reads single line
readlines()	Reads all lines as list

Conclusion

These functions are useful for reading file contents in different ways.

==========

5(b) Describe the Following Terms in Detail

i) self ii) Constructor iii) class variable iv) Instance variable

1. self

`self` refers to current object of the class.

It is used to access instance variables and methods.

Example

```
class Test:
    def show(self):
        print("Hello")
```

---

2. Constructor

Constructor initializes object data automatically.

Python constructor method is `__init__()`

Example

```
class Student:
    def __init__(self, name):
        self.name = name
```

---

3. Class Variable

Shared by all objects.

Example

```
class Student:
    college = "ABC College"
```

---

4. Instance Variable

Unique for each object.

Example

```
class Student:
    def __init__(self, name):
        self.name = name
```

Complete Example

```
class Student:
    college = "ABC College"

    def __init__(self, name):
        self.name = name

    def display(self):
        print(self.name)
        print(Student.college)


s = Student("Rahul")
s.display()
```

Conclusion

These concepts form the foundation of object-oriented programming.

==========

6(a) List and Explain Object-Oriented Concepts

Main OOP Concepts

1. Class
2. Object
3. Encapsulation
4. Inheritance
5. Polymorphism
6. Abstraction

---

1. Class

Blueprint for creating objects.

Example

```
class Student:
    pass
```

---

2. Object

Instance of a class.

Example

```
s = Student()
```

---

3. Encapsulation

Binding data and methods together.

---

4. Inheritance

Acquiring properties from another class.

---

5. Polymorphism

Same method behaves differently.

---

6. Abstraction

Hiding implementation details.

Advantages of OOP

1. Reusability
2. Security
3. Modularity
4. Easy maintenance

Conclusion

OOP concepts help in building efficient and real-world applications.

==========

6(b) What are the Different Modes of Opening a File in Python? Explain the Python open() Built-in Function.

open() Function

`open()` is used to open a file.

Syntax

```
open(filename, mode)
```

File Modes

Mode	Description
r	Read mode
w	Write mode
a	Append mode
x	Create mode
rb	Read binary
wb	Write binary
r+	Read and write

Example

```
file = open("sample.txt", "w")
file.write("Hello")
file.close()
```

Explanation

1. `sample.txt` is filename
2. `w` opens file in write mode
3. `write()` stores data
4. `close()` closes file

Conclusion

Different modes allow different file operations efficiently.

==========

7(a) Write a Python Program to Create a Class ‘student’ with Members {name, branch, grade}. Define Appropriate Member Functions for Reading and Displaying the Student Information.

Program

```
class Student:
    def read(self):
        self.name = input("Enter name: ")
        self.branch = input("Enter branch: ")
        self.grade = input("Enter grade: ")

    def display(self):
        print("Name:", self.name)
        print("Branch:", self.branch)
        print("Grade:", self.grade)
```

```
s = Student()
s.read()
s.display()
```

Explanation

1. `read()` accepts student data
2. `display()` shows details
3. Object `s` calls methods

Conclusion

Classes simplify management of related data and methods.

==========

7(b) Write Python Code Snippets to Illustrate Method Overriding and Method Overloading Concepts

1. Method Overriding

Derived class redefines parent class method.

Example

```
class Parent:
    def show(self):
        print("Parent Method")

class Child(Parent):
    def show(self):
        print("Child Method")


obj = Child()
obj.show()
```

Output

```
Child Method
```

---

2. Method Overloading

Same method performs different tasks with different arguments.

Example

```
class Test:
    def add(self, a, b=0, c=0):
        print(a + b + c)


obj = Test()
obj.add(10, 20)
obj.add(10, 20, 30)
```

Output

```
30
60
```

Conclusion

Method overriding and overloading improve flexibility and polymorphism.

==========

UNIT – 5 : EXCEPTION HANDLING AND GUI PROGRAMMING

1(a) List and Explain Any Four Built-in Error Types in Python

1. ZeroDivisionError

Occurs when division by zero is attempted.

Example

```
print(10/0)
```

---

2. NameError

Occurs when undefined variable is used.

Example

```
print(x)
```

---

3. TypeError

Occurs when incompatible data types are used.

Example

```
print("10" + 5)
```

---

4. IndexError

Occurs when invalid list index is accessed.

Example

```
list = [1,2,3]
print(list[5])
```

Conclusion

Built-in errors help identify runtime problems in programs.

==========

1(b) Compare Terminal Based with GUI-Based Programs

Terminal Based Programs

Programs executed using command line interface.

Features

1. Text-based interaction
2. Faster execution
3. Less memory usage
4. Requires command knowledge

GUI-Based Programs

Programs with graphical interface.

Features

1. User-friendly
2. Uses buttons and windows
3. Easier interaction
4. More memory usage

Comparison Table

Terminal Based	GUI Based
Text interface	Graphical interface
Keyboard dependent	Mouse and keyboard
Less attractive	More attractive
Difficult for beginners	Easy for beginners

Conclusion

GUI programs provide better user interaction compared to terminal-based programs.

==========

2(a) Define Exceptions and How to Handle Exceptions in a Python Program

Definition of Exception

An exception is a runtime error that interrupts normal execution of program.

Exception Handling

Python handles exceptions using:

1. try
2. except
3. else
4. finally

Syntax

```
try:
    statements
except Exception:
    statements
```

Example Program

```
try:
    a = 10
    b = 0
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Output

```
Cannot divide by zero
```

Advantages

1. Prevents program crash
2. Improves reliability
3. Provides error messages

Conclusion

Exception handling improves robustness of Python programs.

==========

2(b) Develop a Python Program to Handle Division by Zero Exception

Program

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))

    result = a / b

    print("Result:", result)
```

```
except ZeroDivisionError:
    print("Division by zero is not allowed")
```

Explanation

1. User inputs two numbers
2. Division operation performed
3. If denominator is zero, exception occurs
4. `except` block handles error

Conclusion

The program prevents abrupt termination using exception handling.

==========

3(a) Demonstrate GUI-Based Program Coding with an Example

GUI Programming

GUI stands for Graphical User Interface.

Python provides Tkinter module for GUI development.

Example Program

```
from tkinter import *

root = Tk()
root.title("My Window")

label = Label(root, text="Welcome to Python GUI")
label.pack()

root.mainloop()
```

Explanation

1. `Tk()` creates window
2. `Label()` creates text label
3. `pack()` arranges widget
4. `mainloop()` runs application

Conclusion

GUI programs provide interactive user interfaces.

==========

3(b) How to Create, Raise and Handle User Defined Exceptions in Python? Illustrate with a Python Program.

User Defined Exception

Custom exceptions created by programmer.

Steps

1. Create exception class
2. Raise exception using `raise`
3. Handle using `try-except`

Example Program

```
class AgeError(Exception):
    pass

try:
    age = int(input("Enter age: "))

    if age < 18:
        raise AgeError

    print("Eligible")
```

```
except AgeError:
    print("Age must be 18 or above")
```

Explanation

1. `AgeError` inherits `Exception`
2. `raise` generates exception
3. `except` handles exception

Conclusion

User-defined exceptions improve program-specific error handling.

==========

4(a) List and Explain Clean-Up Actions with Examples

Definition

Clean-up actions are operations performed before program termination.

Used for releasing resources like files and database connections.

---

finally Block

`finally` block always executes.

Example Program

```
try:
    file = open("data.txt", "r")
    print(file.read())
except FileNotFoundError:
    print("File not found")
```

```
finally:
    print("Program completed")
```

Advantages

1. Releases resources properly
2. Prevents memory leaks
3. Ensures proper execution

Conclusion

Clean-up actions improve reliability and resource management.

==========

4(b) Develop a Python Program to Implement a Single try Block with Multiple except Blocks and Trace the Code for its Execution.

Program

```
try:
    a = int(input("Enter number: "))
    b = int(input("Enter number: "))

    result = a / b

    list = [1, 2, 3]
    print(list[5])

except ZeroDivisionError:
    print("Division by zero")

except IndexError:
    print("Invalid index")

except ValueError:
    print("Invalid input")
```

Explanation

1. `try` block may generate different exceptions
2. Each `except` handles specific exception
3. Program execution continues safely

Trace

- If `b=0` → ZeroDivisionError
- If invalid index → IndexError
- If wrong input type → ValueError

Conclusion

Multiple except blocks improve handling of different runtime errors.

==========

5(a) Is it Possible to Implement Multiple Exception Blocks in Python Exception Handling? Justify Your Answer.

Yes, it is possible.

Python allows multiple `except` blocks after a single `try` block.

This helps in handling different exceptions separately.

Example Program

```
try:
    num = int(input("Enter number: "))
    result = 10 / num

except ZeroDivisionError:
    print("Cannot divide by zero")
```

```
except ValueError:
    print("Invalid input")
```

Advantages

1. Handles different errors separately
2. Improves readability
3. Prevents abnormal termination
4. Provides specific error messages

Conclusion

Multiple exception blocks make exception handling more efficient and reliable.

==========

5(b) Explain the Purpose of ‘else’ and ‘finally’ Blocks in Exception Handling with a Python Program

else Block

`else` executes when no exception occurs.

---

finally Block

`finally` executes whether exception occurs or not.

Example Program

```
try:
    a = int(input("Enter number: "))
    result = 10 / a

except ZeroDivisionError:
    print("Division by zero")
```

```
else:
    print("Result:", result)

finally:
    print("Program ended")
```

Explanation

1. `try` executes risky code
2. `except` handles errors
3. `else` runs if no error occurs
4. `finally` always executes

Advantages

1. Better program control
2. Proper resource management
3. Improves reliability

Conclusion

`else` and `finally` blocks improve exception handling efficiency and program safety.

==========

FINAL EXAM PREPARATION TIPS

1. Practice all programs manually.
2. Learn syntax carefully.
3. Understand definitions and advantages.
4. Write neat diagrams and tables where needed.
5. Mention output for every program.
6. Explain each step clearly in examinations.
7. Use proper indentation in Python programs.
8. Write conclusions for theoretical answers.

IMPORTANT POINT

If these answers are prepared thoroughly along with program practice, a student can confidently score very high marks in the examination.